

CO328 Software Engineering — Week 6: Testing

Course	CO328 Software Engineering
Group	Group 02
Week	06 — Unit Testing, Mocking and CI
Date	July 10, 2026
Project	Drone Swarm (e21-3yp-Drone-Swarm) — module under test: Drone-swarm-v1/computer_code/api

Group members and assigned functions

Index No.	Name	Assigned function (module)
E/21/156	Gunaratne I. A. T. A.	Controller.step() flight state machine (controller.py)
E/21/180	Herath H. M. S. R.	Controller.serialize_state / serialize_pid / serialize_trim (controller.py)
E/21/009	Abeysekera L. B. B.	_detect_bright_spot + threshold validation (tracker.py)
E/21/217	Kaushalya K. A. D. I.	_triangulate_from_detected / _cam1_to_world_mm (tracker.py)

Step 1: Systematic Test Case Design

The functions below were selected because they carry the safety-critical business logic of the PC-side flight supervisor and the vision-based localisation pipeline, and none of them had automated tests. For each function we applied equivalence partitioning (EP), boundary value analysis (BVA) and explicit error/negative cases, and documented the external dependencies that had to be mocked or stubbed to keep the tests isolated.

1. *Controller.step() flight state machine — E/21/156 Gunaratne I. A. T. A.*

Aspect	Design
Behaviour	Per-tick supervisor turning (position, velocity, heading) into the tagged state packet streamed to the sender ESP32. Owns IDLE → ARMING → READY → TAKEOFF → HOVER → LANDING → EMERGENCY transitions.
EP classes	(a) disarmed states IDLE/EMERGENCY → armed=0 safe packet; (b) armed-parked states ARMING/READY → armed=1, setpoint = current position, z_sp=0; (c) flying states TAKEOFF/HOVER/LANDING → armed=2, latched setpoint; (d) pos/vel valid ndarray vs None (fault class).
BVA	arming_hold_s = 0.3 s: exactly 0.3 s stays ARMING (strict >), 0.301 s → READY. Take-off ramp fraction = 1.0: z_sp equals target exactly and state becomes HOVER. Landing fraction = 1.0: z_sp = 0, auto-disarm → IDLE. z_err_limit_m = 0.5 m: error == 0.5 is safe (strict >), 0.65 m sustained > 0.5 s → EMERGENCY; transient violation < 0.5 s recovers.
Error cases	pos=None or vel=None → EMERGENCY safe packet (armed=0, velocities zeroed); cmd_takeoff() while disarmed → ignored, stays IDLE; cmd_arm(False) from any state → IDLE.
Dependencies mocked	time.perf_counter (module-level in controller.py) replaced with a FakeClock fixture advanced manually — makes every timing branch deterministic with no sleeps. No serial port is touched (step() returns a dict; serialisation tested separately).

2. *Serial protocol encoders — E/21/180 Herath H. M. S. R.*

Aspect	Design
--------	--------

Behaviour	serialize_state() encodes the step() dict as one "S,..." line; serialize_pid() encodes exactly 17 PID/ground-effect gains as a "P,..." line; serialize_trim() encodes 4 trim ints as a "T,..." line. These lines are parsed by the sender ESP32 firmware, so the format is a hard contract.
EP classes	valid packet dict → 12-field S-line with 4-dp floats and integer armed flag; numeric gain sequences; int-coercible trim values (bool/float accepted).
BVA	Gain count around the protocol constant 17: len 16 → zero-padded to 17; len 17 → unchanged; len 18 → truncated to 17; len 0 → all-zero line (extreme lower bound).
Error cases	packet missing a key → KeyError; non-numeric gain → ValueError; non-numeric trim → ValueError; floats in trim are truncated toward zero by int().
Dependencies mocked	None — pure functions (static methods), fully isolated by construction.

3. *_detect_bright_spot()* + *threshold validation* — E/21/009 Abeysekera L. B. B.

Aspect	Design
Behaviour	Per-frame LED detection: grayscale → Gaussian blur → binary threshold → morphological open/dilate → largest valid contour centroid. This is the very first stage of the localisation pipeline — every downstream position estimate depends on it. Also covers Tracker.set_threshold(s), the validation layer for the detector's threshold.
EP classes	(a) frame with no bright region → None; (b) one valid LED blob → its centroid; (c) two valid blobs → centroid of the larger one; (d) blob brightness ≤ threshold → treated as no blob.
BVA	Brightness vs threshold=180 (cv.THRESH_BINARY is strict >): uniform blob of value 180 → rejected (exactly on); value 181 → detected (just above). Blob area vs min=3 px: single pixel (erased by blur/morph-open) → None; 5×5 blob → detected. Area vs max=5000 px: 60×60 (~3.8k, just inside) → detected; 80×80 (~6.7k, outside) → None. Threshold validation clip to [0,255]: -5 → 0, 300 → 255, exactly 0/255 kept; short list padded, long list truncated, empty list → default 180.
Error cases	grayscale 2-D frame → cv2.error (cvtColor expects BGR); None frame → cv2.error.
Dependencies mocked	Tracker's calibration-file loaders (_load_intrinsics, _load_extrinsics, _load_world_transform) monkeypatched with synthetic in-memory calibration — no files or USB cameras touched. Frames are synthetic numpy images; cv2 image ops are deterministic pure maths, used real.

4. *_triangulate_from_detected()* / *_cam1_to_world_mm()* — E/21/217 Kaushalya K. A. D. I.

Aspect	Design
Behaviour	Multi-camera triangulation: undistorts the per-camera LED pixels and averages the pairwise cv2 triangulations into one 3-D point, which _cam1_to_world_mm() then maps into the world frame (scale · R · p + t). This pair of functions produces the position the drone actually flies on.
EP classes	detection count: 0 or 1 cameras → None (cannot triangulate); exactly 2 → single-pair estimate; 3+ → mean of all pairwise estimates. Verified against ground truth by projecting a known 3-D point through a synthetic pinhole rig and recovering it to 1e-6; half-pixel noise recovers to ~cm at 2 m.
BVA	The len(detected) < 2 guard: exactly 1 detection → None (just outside); exactly 2 → valid point (just inside). _cam1_to_world_mm: identity transform → point unchanged; known scale/R/t → matches hand-computed affine; scale = 0 (degenerate lower bound) → collapses to translation t.
Error cases	point of length 2 → ValueError (reshape(3,1) fails); non-numeric point → ValueError.
Dependencies mocked	Entire camera rig replaced by synthetic in-memory calibration (K, D=0, known extrinsics via _build_projection) — no calibration files, cameras or threads. cv2.triangulatePoints used real (deterministic).

Supplementary: the two signal-conditioning stages between tracker and controller (KalmanFilter.update/predict_only and LowPassFilter.filter) were additionally tested with the same EP/BVA method (27 further tests, time.perf_counter and LowPassFilter mocked). They are included in the repository suite and CI but the four functions above are the primary submission.

Step 2: Implementation and Local Run Results

Framework: **pytest 9.1.1** with **unittest.mock** and pytest's **monkeypatch** for isolation (Python 3.10, numpy 2.2, scipy 1.15, opencv-python-headless 4.13). Tests live in Drone-swarm-v1/computer_code/api/tests/: a shared `conftest.py` provides the FakeClock fixture, plus one test module per function. All four members' tests were merged into the shared repository and the full suite runs together with no conflicts.

Function under test	Member	Tests written	Passing
Controller.step() state machine	E/21/156 Gunaratne	15	15
serialize_state / serialize_pid / serialize_trim	E/21/180 Herath	12	12
_detect_bright_spot + threshold validation	E/21/009 Abeysekera	17	17
_triangulate_from_detected / _cam1_to_world_mm	E/21/217 Kaushalya	12	12
Supplementary: KalmanFilter + LowPassFilter	group	27	27
Total		83	83

Local test run (pytest -v, condensed)

```
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
rootdir: /sessions/practical-blissful-darwin/mnt/e21-3yp-Drone-Swarm/Drone-swarm-v1/computer_code/api
collecting ... collected 83 items

tests/test_controller_state_machine.py::test_initial_state_is_idle_and_disarmed PASSED [ 1%]
tests/test_controller_state_machine.py::test_idle_packet_setpoints_track_current_position PASSED [ 2%]
tests/test_controller_state_machine.py::test_cmd_takeoff_ignored_when_disarmed PASSED [ 3%]
tests/test_controller_state_machine.py::test_arm_transitions_idle_to_arming_and_latches_xy PASSED [ 4%]
tests/test_controller_state_machine.py::test_arming_hold_boundary_exactly_on_stays_arming PASSED [ 6%]
tests/test_controller_state_machine.py::test_arming_hold_boundary_just_over_goes_ready PASSED [ 7%]
tests/test_controller_state_machine.py::test_takeoff_ramp_reaches_target_exactly_then_hover PASSED [ 8%]
tests/test_controller_state_machine.py::test_takeoff_ramp_midpoint_is_half_target PASSED [ 9%]
tests/test_controller_state_machine.py::test_landing_completes_disarms_and_returns_idle PASSED [ 10%]
tests/test_controller_state_machine.py::test_z_error_exactly_at_limit_is_safe PASSED [ 12%]
tests/test_controller_state_machine.py::test_sustained_z_error_over_limit_triggers_emergency PASSED [ 13%]
tests/test_controller_state_machine.py::test_transient_z_error_recovers_without_emergency PASSED [ 14%]
tests/test_controller_state_machine.py::test_missing_pose_forces_emergency_safe_packet PASSED [ 15%]
tests/test_controller_state_machine.py::test_missing_velocity_forces_emergency PASSED [ 16%]
tests/test_controller_state_machine.py::test_disarm_from_any_state_returns_idle PASSED [ 18%]
tests/test_kalman_filter.py::test_first_update_returns_measurement_and_zero_velocity PASSED [ 19%]
tests/test_kalman_filter.py::test_accepts_list_tuple_and_ndarray PASSED [ 20%]
tests/test_kalman_filter.py::test_stationary_target_converges_to_measurement PASSED [ 21%]
tests/test_kalman_filter.py::test_velocity_estimated_from_states_with_identity_lpf PASSED [ 22%]
tests/test_kalman_filter.py::test_predict_only_before_init_returns_zeros PASSED [ 24%]
tests/test_kalman_filter.py::test_predict_only_extrapolates_position PASSED [ 25%]
tests/test_kalman_filter.py::test_dt_clamped_up_to_1ms PASSED [ 26%]
tests/test_kalman_filter.py::test_dt_clamped_down_to_200ms PASSED [ 27%]
tests/test_kalman_filter.py::test_dt_inside_range_used_exactly PASSED [ 28%]
tests/test_kalman_filter.py::test_transition_matrix_kinematics PASSED [ 30%]
tests/test_kalman_filter.py::test_wrong_measurement_length_raises[bad0] PASSED [ 31%]
tests/test_kalman_filter.py::test_wrong_measurement_length_raises[bad1] PASSED [ 32%]
tests/test_kalman_filter.py::test_wrong_measurement_length_raises[bad2] PASSED [ 33%]
tests/test_kalman_filter.py::test_non_numeric_measurement_raises PASSED [ 34%]
tests/test_kalman_filter.py::test_reset_returns_filter_to_uninitialised PASSED [ 36%]
tests/test_low_pass_filter.py::test_output_shape_matches_dims PASSED [ 37%]
tests/test_low_pass_filter.py::test_constant_input_converges_to_input PASSED [ 38%]
tests/test_low_pass_filter.py::test_single_dim_filter_works PASSED [ 39%]
tests/test_low_pass_filter.py::test_attenuates_high_frequency_noise PASSED [ 40%]
tests/test_low_pass_filter.py::test_buffer_just_inside_not_trimmed PASSED [ 42%]
tests/test_low_pass_filter.py::test_buffer_at_size_trimmed_to_half PASSED [ 43%]
tests/test_low_pass_filter.py::test_output_continuous_across_trim PASSED [ 44%]
tests/test_low_pass_filter.py::test_cutoff_just_below_nyquist_valid PASSED [ 45%]
tests/test_low_pass_filter.py::test_cutoff_at_nyquist_raises PASSED [ 46%]
tests/test_low_pass_filter.py::test_cutoff_zero_raises PASSED [ 48%]
tests/test_low_pass_filter.py::test_wrong_dims_sample_raises PASSED [ 49%]
tests/test_low_pass_filter.py::test_non_numeric_sample_raises PASSED [ 50%]
tests/test_serializers.py::test_state_line_format_and_field_count PASSED [ 51%]
tests/test_serializers.py::test_state_line_values_are_4dp_floats PASSED [ 53%]
tests/test_serializers.py::test_state_line_armed_boolean_coercion PASSED [ 54%]
tests/test_serializers.py::test_state_missing_key_raises_keyerror PASSED [ 55%]
tests/test_serializers.py::test_pid_exactly_17_passed_through PASSED [ 56%]
tests/test_serializers.py::test_pid_16_padded_to_17_with_zero PASSED [ 57%]
tests/test_serializers.py::test_pid_18_truncated_to_17 PASSED [ 59%]
tests/test_serializers.py::test_pid_empty_list_gives_all_zeros PASSED [ 60%]
```

```
tests/test_serializers.py::test_pid_non_numeric_raises_valueerror PASSED [ 61%]
tests/test_serializers.py::test_trim_line_format PASSED [ 62%]
tests/test_serializers.py::test_trim_floats_truncated_to_int PASSED [ 63%]
tests/test_serializers.py::test_trim_non_numeric_raises_valueerror PASSED [ 65%]
tests/test_tracker_detection.py::test_empty_frame_returns_none PASSED [ 66%]
tests/test_tracker_detection.py::test_single_led_blob_detected_near_centre PASSED [ 67%]
tests/test_tracker_detection.py::test_larger_of_two_blobs_wins PASSED [ 68%]
tests/test_tracker_detection.py::test_brightness_exactly_at_threshold_rejected PASSED [ 69%]
tests/test_tracker_detection.py::test_brightness_one_above_threshold_detected PASSED [ 71%]
tests/test_tracker_detection.py::test_single_pixel_below_min_area_rejected PASSED [ 72%]
tests/test_tracker_detection.py::test_small_valid_blob_detected PASSED [ 73%]
tests/test_tracker_detection.py::test_blob_inside_max_area_detected PASSED [ 74%]
tests/test_tracker_detection.py::test_blob_above_max_area_rejected PASSED [ 75%]
tests/test_tracker_detection.py::test_grayscale_frame_raises_cv_error PASSED [ 77%]
tests/test_tracker_detection.py::test_none_frame_raises_cv_error PASSED [ 78%]
tests/test_tracker_detection.py::test_set_threshold_clips_above_255 PASSED [ 79%]
tests/test_tracker_detection.py::test_set_threshold_clips_below_0 PASSED [ 80%]
tests/test_tracker_detection.py::test_set_threshold_boundary_values_kept PASSED [ 81%]
tests/test_tracker_detection.py::test_set_thresholds_short_list_padded_with_last PASSED [ 83%]
tests/test_tracker_detection.py::test_set_thresholds_long_list_truncated PASSED [ 84%]
tests/test_tracker_detection.py::test_set_thresholds_empty_falls_back_to_default PASSED [ 85%]
tests/test_tracker_triangulation.py::test_no_detections_returns_none PASSED [ 86%]
tests/test_tracker_triangulation.py::test_single_detection_returns_none PASSED [ 87%]
tests/test_tracker_triangulation.py::test_two_cameras_recover_ground_truth PASSED [ 89%]
tests/test_tracker_triangulation.py::test_three_cameras_recover_ground_truth PASSED [ 90%]
tests/test_tracker_triangulation.py::test_three_cameras_average_pairwise_estimates PASSED [ 91%]
tests/test_tracker_triangulation.py::test_noisy_pixels_give_approximate_point PASSED [ 92%]
tests/test_tracker_triangulation.py::test_identity_transform_leaves_point_unchanged PASSED [ 93%]
tests/test_tracker_triangulation.py::test_known_scale_rotation_translation PASSED [ 95%]
tests/test_tracker_triangulation.py::test_zero_scale_collapses_to_translation PASSED [ 96%]
tests/test_tracker_triangulation.py::test_output_shape_is_flat_3vector PASSED [ 97%]
tests/test_tracker_triangulation.py::test_wrong_length_point_raises PASSED [ 98%]
tests/test_tracker_triangulation.py::test_non_numeric_point_raises PASSED [100%]
```

```
===== 83 passed in 0.53s =====
```

Step 3: CI Automation and Group Test Review

GitHub Actions workflow

The workflow below was added at `.github/workflows/tests.yml` (repository root). It triggers on every push and pull request, installs the pinned scientific dependencies (opencv-python-headless replaces opencv-python on CI — same cv2 API, no GUI libraries), runs the full suite, and the job fails if any test fails, so a red status blocks a broken merge.

```
# C0328 Week 6 - CI workflow
# Runs the computer_code API unit test suite on every push and pull request.
# The job fails if any test fails, blocking a broken merge.

name: Unit Tests

on:
  push:
  pull_request:

jobs:
  test:
    name: pytest (computer_code API)
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: Drone-swarm-v1/computer_code/api

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip

      - name: Install dependencies
        # opencv-python-headless: same cv2 API, no GUI libs needed on CI
        run: |
          python -m pip install --upgrade pip
          pip install numpy scipy opencv-python-headless pytest

      - name: Run test suite
        run: pytest tests -v --tb=short
```

[Screenshot placeholder — after pushing this commit, insert a screenshot of the green "Unit Tests" run from the repository's Actions tab here before uploading to the LMS.]

Peer test review (functions swapped between members)

Each member reviewed another member's function and tried to find one input that could still break the code but is not covered by any existing test:

Function	Reviewer	Gap found (uncovered breaking input)
Controller.step()	E/21/180 Herath	A NaN position (e.g. tracker glitch producing [nan, nan, nan]) passes the <code>pos is None</code> guard: NaN propagates into the setpoints and the z-error EMERGENCY check (<code>nan > 0.5</code> is False), so the failsafe never fires. No test covers NaN poses.
Serializers	E/21/009 Abeysekera	<code>serialize_state()</code> with inf/NaN values happily emits "nan" / "inf" in the S-line, which the ESP32's float parser would misread. No test asserts rejection of non-finite values.
<code>_detect_bright_spot</code>	E/21/217 Kaushalya	Two blobs of exactly equal area: the <code>strict_area > best_area</code> comparison silently keeps whichever contour OpenCV happens to return first — with two drones in frame the tracked target could flip between them. No test covers equal-area ambiguity, nor a fully saturated frame (sunlit room) where the single merged contour exceeds <code>max_area</code> and the LED is reported lost.

<code>_triangulate_from_detected</code>	E/21/156 Gunaratne	Two cameras with (near-)identical projections or matching pixels on a degenerate baseline make <code>cv2.triangulatePoints</code> divide by $w \approx 0$, returning <code>inf/NaN</code> which is averaged into the result unguarded — a non-finite world position would flow straight into the Kalman filter. No test asserts the output is finite.
---	--------------------	--

What the suite does catch: every state transition and its timing boundary, the disarm/failsafe paths, the serial protocol's 17-gain contract, the detector's brightness/area boundaries and threshold clipping, and geometric correctness of triangulation against a known ground truth. **What it does not catch (agreed follow-ups):** non-finite values (NaN/inf) end-to-end through the pipeline, multi-blob/equal-area ambiguity, degenerate camera geometry, and concurrency (`step()` called from two threads). These were added to the Trello board as "Week 7: harden inputs" cards.

Test code

Full test sources are committed under `Drone-swarm-v1/computer_code/api/tests/` (`conftest.py`, `test_controller_state_machine.py`, `test_serializers.py`, `test_tracker_detection.py`, `test_tracker_triangulation.py`, plus supplementary `test_kalman_filter.py` and `test_low_pass_filter.py`). Key excerpts:

```
# conftest.py -- deterministic clock injected in place of time.perf_counter
class FakeClock:
    def __init__(self, start=1000.0): self.t = start
    def __call__(self): return self.t
    def advance(self, seconds): self.t += seconds

# test_controller_state_machine.py -- BVA on the 0.3 s arming hold (strict >)
def test_arming_hold_boundary_exactly_on_stays_arming(ctl):
    c, clock = ctl
    c.cmd_arm(True)
    c.step(POS0, VEL0, 0.0, DT)          # -> ARMING at t0
    clock.advance(0.3)                   # exactly arming_hold_s
    c.step(POS0, VEL0, 0.0, DT)
    assert c.get_state() == "ARMING"

# test_tracker_detection.py -- calibration loaders mocked, no files/cameras
@pytest.fixture
def tracker_no_files(monkeypatch):
    monkeypatch.setattr(tracker_mod, "_load_intrinsics",
                        lambda d: [{"K": K, "D": D} for _ in range(4)])
    monkeypatch.setattr(tracker_mod, "_load_extrinsics",
                        lambda d: [{"R_cam_to_cam1": np.eye(3),
                                    "C_cam1": np.zeros((3, 1))} for _ in range(4)])
    monkeypatch.setattr(tracker_mod, "_load_world_transform",
                        lambda d: (1.0, np.eye(3), np.zeros((3, 1))))
    return Tracker()

# test_tracker_detection.py -- BVA on brightness (cv.THRESH_BINARY strict >)
def test_brightness_exactly_at_threshold_rejected():
    f = add_blob(black_frame(), 320, 240, 21, value=THRESH)    # == 180
    assert detect(f) is None

# test_tracker_triangulation.py -- ground truth via a synthetic pinhole rig
def test_two_cameras_recover_ground_truth():
    pt = _triangulate_from_detected(detections([0, 1]), INTRINSICS, PROJECTIONS)
    np.testing.assert_allclose(pt, GT_POINT, atol=1e-6)

# test_serializers.py -- boundary on the 17-gain protocol constant
def test_pid_18_truncated_to_17():
    gains = [1.0] * 17 + [99.0]
    fields = _pid_fields(gains)
    assert len(fields) == 18 and "99.000000" not in fields
```